Activity 3.3 K Nearest Neighbors

Objective(s):

This activity aims to perform classification using K Nearest Neighbors

Intended Learning Outcomes (ILOs):

- Build the model using K Nearest Neighbors.
- Evaluate the performance of the model.
- Choose the best k-value.

Resources:

- Jupyter Notebook
- Classified data

Procedure:

Import the libraries

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline
```

Load the data. Set index_col=0 to use the first column as the index.

```
df = pd.read_csv("classified_data", index_col=0)

df.head()
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 1000,\n  \"fields\":
[\n    {\n      \"column\": \"WTT\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.2896352516575889,\n
\"min\": 0.174411668391638,\n        \"max\": 1.721779168965468,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
0.7063010303254464,\n          1.3496574995895918,\n
1.3169005830819778\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"PTI\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.2570852621379548,\n        \"min\": 0.441398100295989,\n
\"max\": 1.8337565522536252,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          1.521177618895161,\n
1.2203336025956588,\n          1.1486493499359351\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"EQW\",\n      \"properties\": {\n
```

```
\"dtype\": \"number\",\n        \"std\": 0.29155385034441145,\n
\"min\": 0.1709236280526556,\n        \"max\": 1.7227247553711322,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
1.130807438901416,\n          0.7058621252266584,\n
1.0433060732882415\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"SBI\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.229645024161446,\n        \"min\": 0.0450266664094166,\n
\"max\": 1.634884045436437,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          0.4034386350051173,\n
0.8143574265988366,\n          0.9019193027640824\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"LQE\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.24341295346925404,\n
\"min\": 0.3153070077960995,\n        \"max\": 1.650049589008639,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
0.8967456416302068,\n          0.6699170570739925,\n
0.9443602753750552\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"QWG\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.256120596610873,\n        \"min\": 0.2623888468883443,\n
\"max\": 1.6669023520657231,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          0.9158571059683956,\n
0.6719997779429592,\n          1.1087710280109884\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"FDJ\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.25511802913126275,\n
\"min\": 0.2952280855806717,\n        \"max\": 1.7133422293242386,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
1.0701452103715168,\n          0.7467547171292814,\n
0.3132418138897407\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"PJF\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.288981643277679,\n        \"min\": 0.299475657020008,\n
\"max\": 1.7854196250383634,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          1.2150324734826512,\n
0.5407172165654959,\n          0.8107025990833856\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"HQE\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.29373751661647246,\n
\"min\": 0.3651566098613977,\n        \"max\": 1.8856900849797629,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
1.0385048132735202,\n          1.4319928464192602,\n
1.0343550687529062\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"NXJ\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.20422502341003687,\n        \"min\": 0.639692747423801,\n
\"max\": 1.8939496030653464,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          1.4457974108545717,\n
```

```
1.173152878750532,\n          1.5851535299635755\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"TARGET CLASS\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0,\n        \"min\": 0,\n        \"max\": 1,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          0,\n
1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


Because the KNN classifier predicts the class of a given test observation by identifying the observations that are nearest to it, the scale of the variables matters. Any variables that are on a large scale will have a much larger effect on the distance between the observations, and hence on the KNN classifier, than variables that are on a small scale.

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

scaler.fit(df.drop('TARGET CLASS',axis=1))

StandardScaler()

scaled_features = scaler.transform(df.drop('TARGET CLASS',axis=1))

df_feat = pd.DataFrame(scaled_features,columns=df.columns[:-1])
df_feat.head()
```


```
{"summary":"{\n  \"name\": \"df_feat\",\n  \"rows\": 1000,\n
\"fields\": [\n    {\n      \"column\": \"WTT\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1.0005003753127737,\n        \"min\": -2.678050294892763,\n
\"max\": 2.667092453480776,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          -0.8407204008480645,\n
1.3816554504162177,\n          1.2685017266467709\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"PTI\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1.0005003753127737,\n
\"min\": -2.6187465594530646,\n        \"max\": 2.799903822014538,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
1.5834383679497588,\n          0.41264176640541583,\n
0.13366769728284045\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"EQW\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1.0005003753127737,\n        \"min\": -2.2758578511888814,\n
\"max\": 3.049325158601135,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          1.0180932040839803,\n          -
0.4401552303536436,\n          0.7178222625994329\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"SBI\",\n      \"properties\": {\n
```

```
\"dtype\": \"number\",\n        \"std\": 1.0005003753127744,\n
\"min\": -2.775551017934635,\n        \"max\": 4.151021461563324,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n          -
1.2140483883610458,\n          0.576212065632318,\n
0.9576951405925229\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"LQE\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1.0005003753127744,\n        \"min\": -2.947205948503044,\n
\"max\": 2.538987100864661,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          -0.5573184627252994,\n          -
1.4896520988184299,\n          -0.3616080116045524\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"QWG\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1.0005003753127737,\n
\"min\": -2.660802498981584,\n        \"max\": 2.8257390273790977,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n          -
0.1081173689453796,\n          -1.060712948686669,\n
0.6454747043228094\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"FDJ\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1.000500375312774,\n        \"min\": -2.6204660364684953,\n
\"max\": 2.940974438292232,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          0.4185386031864608,\n          -
0.8497069652008912,\n          -2.5498213156881664\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"PJF\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1.0005003753127735,\n
\"min\": -2.6744652897720127,\n        \"max\": 2.4701088805242493,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
0.49533791330152793,\n          -1.8392486714127034,\n          -
0.9045163698230286\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"HQE\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1.0005003753127735,\n        \"min\": -2.7013608425848443,\n
\"max\": 2.4777335068709863,\n        \"num_unique_values\": 1000,\n
\"samples\": [\n          -0.4078671888285603,\n
0.9323937672052752,\n          -0.422001648049673\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"NXJ\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1.0005003753127732,\n
\"min\": -3.542140064466624,\n        \"max\": 2.6024764661358253,\n
\"num_unique_values\": 1000,\n        \"samples\": [\n
0.40697451922755945,\n          -0.9287136944022932,\n
1.0896815016660177\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df_feat"}
```


Split the data into a training set and a testing set

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test =
train_test_split(scaled_features,df['TARGET CLASS'],
                                                    test_size=0.30)
```

Buid a model using k=1

Remember that wwe are building a model to predict whether someone will TARGET CLASS or not.

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=1)

knn.fit(X_train,y_train)

KNeighborsClassifier(n_neighbors=1)

pred = knn.predict(X_test)
```

Evaluate the KNN model.

```
from sklearn.metrics import classification_report,confusion_matrix,
accuracy_score

print(confusion_matrix(y_test,pred))
```

```
[[128  11]
 [ 15 146]]
```

```
print(classification_report(y_test,pred))
```

```
              precision    recall  f1-score   support

           0       0.90      0.92      0.91       139
           1       0.93      0.91      0.92       161

    accuracy                           0.91       300
   macro avg       0.91      0.91      0.91       300
weighted avg       0.91      0.91      0.91       300
```

Choose a K value. Create a method to pick a good value of K.

```
error_rate = []


for i in range(1,40):

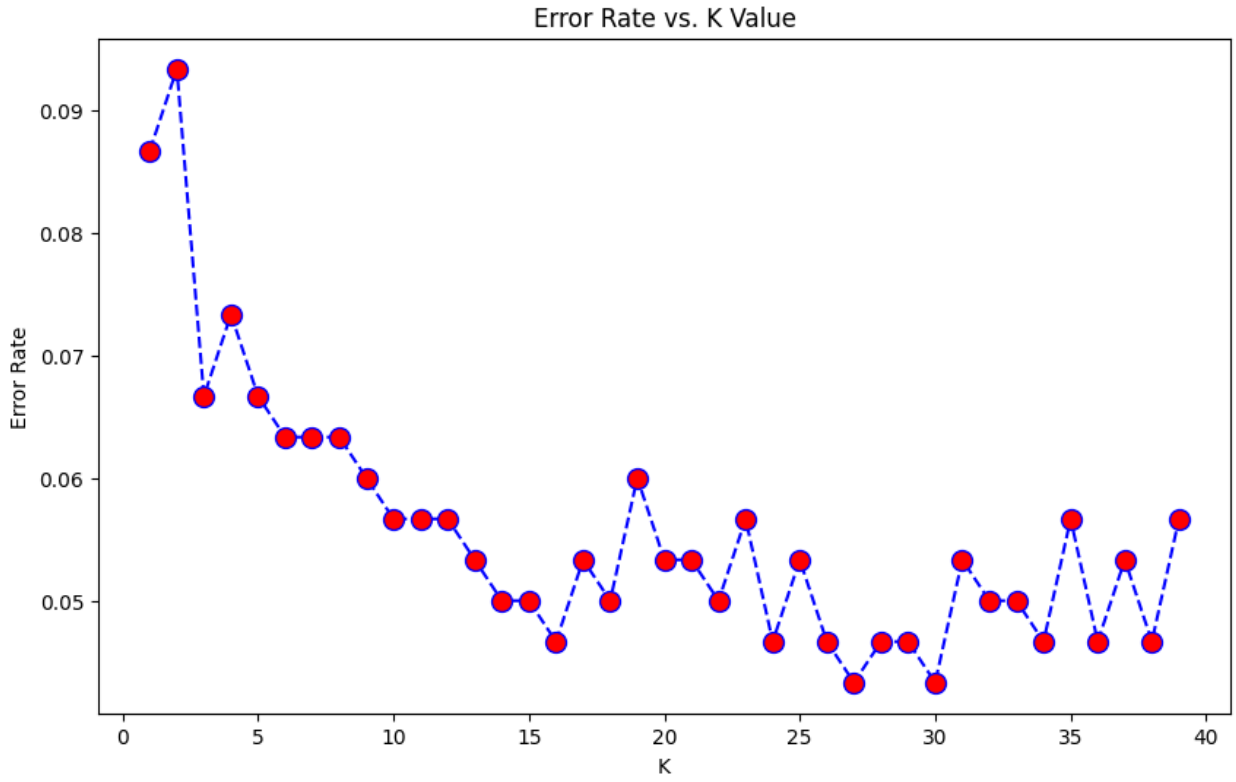
    knn = KNeighborsClassifier(n_neighbors=i)
```

```
    knn.fit(X_train,y_train)
    pred_i = knn.predict(X_test)
    error_rate.append(np.mean(pred_i != y_test))
```

Create a visualization to compare the error rate and k value

```
plt.figure(figsize=(10,6))
plt.plot(range(1,40),error_rate,color='blue', linestyle='dashed',
marker='o',
         markerfacecolor='red', markersize=10)
plt.title('Error Rate vs. K Value')
plt.xlabel('K')
plt.ylabel('Error Rate')
```

```
Text(0, 0.5, 'Error Rate')
```



Interpret the result of the graph

Increasing the number of k or selected neighbors will decrease the error rate of the model. Based on the graph, setting k from 26-30 provides the lowest error rates and having k values that are too low or high leads to inconsistent results with varying error rates.

Compare the classification report of K=1 and K=23

```
# K = 1
knn = KNeighborsClassifier(n_neighbors=1)

knn.fit(X_train,y_train)
pred = knn.predict(X_test)

print('WITH K=1')
print('\n')
print(confusion_matrix(y_test,pred))
print('\n')
print(classification_report(y_test,pred))
```

```
WITH K=1


[[128  11]
 [ 15 146]]


              precision    recall  f1-score   support

           0       0.90      0.92      0.91       139
           1       0.93      0.91      0.92       161

    accuracy                           0.91       300
   macro avg       0.91      0.91      0.91       300
weighted avg       0.91      0.91      0.91       300
```

```
#K = 23
knn = KNeighborsClassifier(n_neighbors=23)

knn.fit(X_train,y_train)
pred = knn.predict(X_test)

print('WITH K=23')
print('\n')
print(confusion_matrix(y_test,pred))
print('\n')
print(classification_report(y_test,pred))
```

```
WITH K=23


[[129  10]
 [  7 154]]


              precision    recall  f1-score   support

           0       0.95      0.93      0.94       139
```

```
           1       0.94      0.96      0.95       161

    accuracy                           0.94       300
   macro avg       0.94      0.94      0.94       300
weighted avg       0.94      0.94      0.94       300
```

Interpret the result of the graph.

Setting k to 23 provide better accuracy than setting k to 1. Higher k values mean more points, which provides more points to depend or refer to when classifying a specific data point.

Fit the K-nearest neighbors model again with `n_neighbors=3` but this time use distance for the weights. Calculate the accuracy using the function you created above.

```
knn = KNeighborsClassifier(n_neighbors=3, weights='distance')

knn = knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print(accuracy_score(y_test, y_pred))
```

```
0.9333333333333333
```

Fit another K-nearest neighbors model. This time use uniform weights but set the power parameter for the Minkowski distance metric to be 1 (`p=1`) i.e. Manhattan Distance.

```
knn = KNeighborsClassifier(n_neighbors=5, p=1)

knn = knn.fit(X_train, y_train)

y_pred = knn.predict(X_test)

print(accuracy_score(y_test, y_pred))
```

```
0.9366666666666666
```

Fit a K-nearest neighbors model using values of `k` (`n_neighbors`) ranging from 1 to 20. Use uniform weights (the default). The coefficient for the Minkowski distance (`p`) can be set to either 1 or 2--just be consistent. Store the accuracy and the value of `k` used from each of these fits in a list or dictionary. Plot (or view the table of) the `accuracy` vs `k`. What do you notice happens when `k=1`?

```
# Fit the K-nearest neighbors model with different values of k
# Store the accuracy measurement for each k

score_list = list()

for k in range(1, 30):
```

```
    knn = KNeighborsClassifier(n_neighbors=k)
    knn = knn.fit(X_train, y_train)

    y_pred = knn.predict(X_test)
    score = accuracy_score(y_test, y_pred)

    score_list.append((k, score))

score_df = pd.DataFrame(score_list, columns=['k', 'accuracy'])

# Import libraries to make the plot

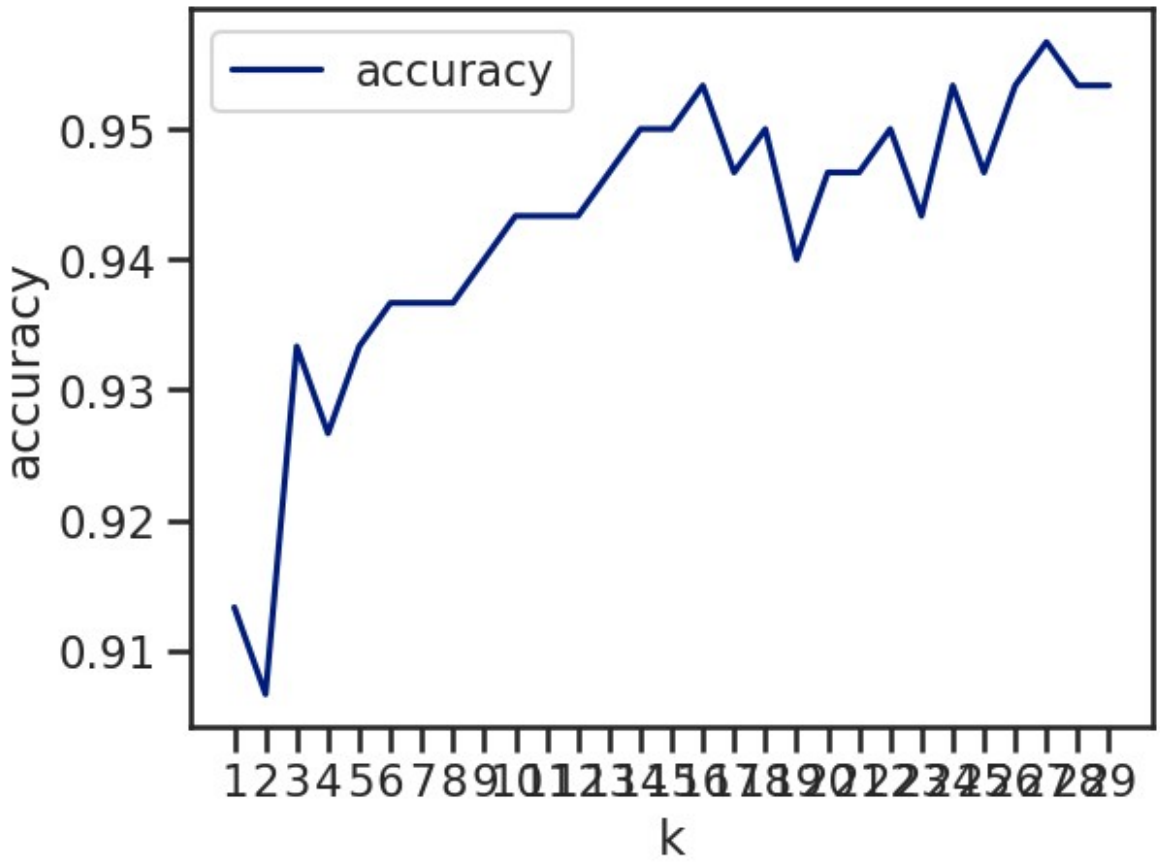
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline

sns.set_context('talk')
sns.set_style('ticks')
sns.set_palette('dark')

ax = score_df.set_index('k').plot()

ax.set(xlabel='k', ylabel='accuracy')
ax.set_xticks(range(1, 30));
```



Supplementary Activities:

Supplementary Activity:

- Choose your own dataset
- Import the dataset
- Determine the number of datapoints, columns and data types
- Remove unneccesary columns
- Do data cleaning such as removing empty values(NaN), replacing missing data .
- Standardize the variables.
- Perform descriptive statistics such as mean, median and mode
- Perform data visualization
- Build the model using K Nearest Neighbor. Use k value from 1 to 50.
- Evaluate the model using classification report, accuracy , confusion matrix and roc_auc_score
- Compare the model using their accuracy report.
- Create a graph to compare the accuracy based from eah k values.

#Dataset: GTZAN Dataset - Music Genre Classification

```
# Import libraries
import pandas as pd
```

```
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
%matplotlib inline

import kagglehub
from kagglehub import KaggleDatasetAdapter

# Set path
file_path = "Data/features_3_sec.csv"

# Load the latest version
df = kagglehub.dataset_load(
  KaggleDatasetAdapter.PANDAS,
  "andradaolteanu/gtzan-dataset-music-genre-classification",
  file_path,
  # Provide any additional arguments like
  # sql_query or pandas_kwargs. See the
  # documentation for more information:
  #
https://github.com/Kaggle/kagglehub/blob/main/README.md#kaggledataseta
dapterpandas
)

df.head()
```

```
Using Colab cache for faster access to the 'gtzan-dataset-music-genre-
classification' dataset.
```

```
{"type":"dataframe","variable_name":"df"}
```

```
df.iloc[0]
```

```
filename                   blues.00000.0.wav
length                                 66149
chroma_stft_mean                    0.335406
chroma_stft_var                     0.091048
rms_mean                            0.130405
rms_var                             0.003521
spectral_centroid_mean           1773.065032
spectral_centroid_var          167541.630869
spectral_bandwidth_mean          1972.744388
spectral_bandwidth_var         117335.771563
rolloff_mean                     3714.560359
rolloff_var                   1080789.885581
zero_crossing_rate_mean             0.081851
zero_crossing_rate_var              0.000558
harmony_mean                       -0.000078
harmony_var                         0.008354
perceptr_mean                      -0.000068
perceptr_var                        0.005535
```

```
tempo                          129.199219
mfcc1_mean                    -118.627914
mfcc1_var                     2440.286621
mfcc2_mean                     125.083626
mfcc2_var                      260.956909
mfcc3_mean                     -23.443724
mfcc3_var                      364.081726
mfcc4_mean                      41.321484
mfcc4_var                      181.694855
mfcc5_mean                      -5.976108
mfcc5_var                      152.963135
mfcc6_mean                      20.115141
mfcc6_var                       75.652298
mfcc7_mean                      -16.04541
mfcc7_var                       40.227104
mfcc8_mean                      17.855198
mfcc8_var                       84.320282
mfcc9_mean                     -14.633434
mfcc9_var                       83.437233
mfcc10_mean                     10.270527
mfcc10_var                      97.001335
mfcc11_mean                     -9.708279
mfcc11_var                      66.669891
mfcc12_mean                     10.183875
mfcc12_var                      45.103611
mfcc13_mean                     -4.681614
mfcc13_var                      34.169498
mfcc14_mean                      8.417439
mfcc14_var                      48.269444
mfcc15_mean                     -7.233477
mfcc15_var                      42.770947
mfcc16_mean                     -2.853603
mfcc16_var                      39.687145
mfcc17_mean                      -3.24128
mfcc17_var                      36.488243
mfcc18_mean                      0.722209
mfcc18_var                      38.099152
mfcc19_mean                     -5.050335
mfcc19_var                      33.618073
mfcc20_mean                     -0.243027
mfcc20_var                      43.771767
label                               blues
Name: 0, dtype: object

# Determine the number of datapoints, columns and data types
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9990 entries, 0 to 9989
Data columns (total 60 columns):
```

```
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   filename                 9990 non-null   object
 1   length                   9990 non-null   int64
 2   chroma_stft_mean         9990 non-null   float64
 3   chroma_stft_var          9990 non-null   float64
 4   rms_mean                 9990 non-null   float64
 5   rms_var                  9990 non-null   float64
 6   spectral_centroid_mean   9990 non-null   float64
 7   spectral_centroid_var    9990 non-null   float64
 8   spectral_bandwidth_mean  9990 non-null   float64
 9   spectral_bandwidth_var   9990 non-null   float64
 10  rolloff_mean             9990 non-null   float64
 11  rolloff_var              9990 non-null   float64
 12  zero_crossing_rate_mean  9990 non-null   float64
 13  zero_crossing_rate_var   9990 non-null   float64
 14  harmony_mean             9990 non-null   float64
 15  harmony_var              9990 non-null   float64
 16  perceptr_mean            9990 non-null   float64
 17  perceptr_var             9990 non-null   float64
 18  tempo                    9990 non-null   float64
 19  mfcc1_mean               9990 non-null   float64
 20  mfcc1_var                9990 non-null   float64
 21  mfcc2_mean               9990 non-null   float64
 22  mfcc2_var                9990 non-null   float64
 23  mfcc3_mean               9990 non-null   float64
 24  mfcc3_var                9990 non-null   float64
 25  mfcc4_mean               9990 non-null   float64
 26  mfcc4_var                9990 non-null   float64
 27  mfcc5_mean               9990 non-null   float64
 28  mfcc5_var                9990 non-null   float64
 29  mfcc6_mean               9990 non-null   float64
 30  mfcc6_var                9990 non-null   float64
 31  mfcc7_mean               9990 non-null   float64
 32  mfcc7_var                9990 non-null   float64
 33  mfcc8_mean               9990 non-null   float64
 34  mfcc8_var                9990 non-null   float64
 35  mfcc9_mean               9990 non-null   float64
 36  mfcc9_var                9990 non-null   float64
 37  mfcc10_mean              9990 non-null   float64
 38  mfcc10_var               9990 non-null   float64
 39  mfcc11_mean              9990 non-null   float64
 40  mfcc11_var               9990 non-null   float64
 41  mfcc12_mean              9990 non-null   float64
 42  mfcc12_var               9990 non-null   float64
 43  mfcc13_mean              9990 non-null   float64
 44  mfcc13_var               9990 non-null   float64
 45  mfcc14_mean              9990 non-null   float64
 46  mfcc14_var               9990 non-null   float64
```

```
 47  mfcc15_mean              9990 non-null   float64
 48  mfcc15_var               9990 non-null   float64
 49  mfcc16_mean              9990 non-null   float64
 50  mfcc16_var               9990 non-null   float64
 51  mfcc17_mean              9990 non-null   float64
 52  mfcc17_var               9990 non-null   float64
 53  mfcc18_mean              9990 non-null   float64
 54  mfcc18_var               9990 non-null   float64
 55  mfcc19_mean              9990 non-null   float64
 56  mfcc19_var               9990 non-null   float64
 57  mfcc20_mean              9990 non-null   float64
 58  mfcc20_var               9990 non-null   float64
 59  label                    9990 non-null   object
dtypes: float64(57), int64(1), object(2)
memory usage: 4.6+ MB

# Explore each column
df["label"].unique()

array(['blues', 'classical', 'country', 'disco', 'hiphop', 'jazz',
       'metal', 'pop', 'reggae', 'rock'], dtype=object)

df["label"].unique()

array(['blues', 'classical', 'country', 'disco', 'hiphop', 'jazz',
       'metal', 'pop', 'reggae', 'rock'], dtype=object)

# Remove unneccesary columns
dfc = df.copy()

dfc.drop(['label', 'filename', 'length'], axis=1, inplace=True)

dfc.columns

Index(['chroma_stft_mean', 'chroma_stft_var', 'rms_mean', 'rms_var',
       'spectral_centroid_mean', 'spectral_centroid_var',
       'spectral_bandwidth_mean', 'spectral_bandwidth_var',
'rolloff_mean',
       'rolloff_var', 'zero_crossing_rate_mean',
'zero_crossing_rate_var',
       'harmony_mean', 'harmony_var', 'perceptr_mean', 'perceptr_var',
'tempo',
       'mfcc1_mean', 'mfcc1_var', 'mfcc2_mean', 'mfcc2_var',
'mfcc3_mean',
       'mfcc3_var', 'mfcc4_mean', 'mfcc4_var', 'mfcc5_mean',
'mfcc5_var',
       'mfcc6_mean', 'mfcc6_var', 'mfcc7_mean', 'mfcc7_var',
'mfcc8_mean',
       'mfcc8_var', 'mfcc9_mean', 'mfcc9_var', 'mfcc10_mean',
'mfcc10_var',
       'mfcc11_mean', 'mfcc11_var', 'mfcc12_mean', 'mfcc12_var',
```

```
'mfcc13_mean',
       'mfcc13_var', 'mfcc14_mean', 'mfcc14_var', 'mfcc15_mean',
'mfcc15_var',
       'mfcc16_mean', 'mfcc16_var', 'mfcc17_mean', 'mfcc17_var',
'mfcc18_mean',
       'mfcc18_var', 'mfcc19_mean', 'mfcc19_var', 'mfcc20_mean',
'mfcc20_var'],
      dtype='object')
```

```
# Removing rows with missing values
dfc.dropna(inplace=True)
dfc.isna().sum()
```

```
chroma_stft_mean           0
chroma_stft_var            0
rms_mean                   0
rms_var                    0
spectral_centroid_mean     0
spectral_centroid_var      0
spectral_bandwidth_mean    0
spectral_bandwidth_var     0
rolloff_mean               0
rolloff_var                0
zero_crossing_rate_mean    0
zero_crossing_rate_var     0
harmony_mean               0
harmony_var                0
perceptr_mean              0
perceptr_var               0
tempo                      0
mfcc1_mean                 0
mfcc1_var                  0
mfcc2_mean                 0
mfcc2_var                  0
mfcc3_mean                 0
mfcc3_var                  0
mfcc4_mean                 0
mfcc4_var                  0
mfcc5_mean                 0
mfcc5_var                  0
mfcc6_mean                 0
mfcc6_var                  0
mfcc7_mean                 0
mfcc7_var                  0
mfcc8_mean                 0
mfcc8_var                  0
mfcc9_mean                 0
mfcc9_var                  0
mfcc10_mean                0
mfcc10_var                 0
```

```
mfcc11_mean                0
mfcc11_var                 0
mfcc12_mean                0
mfcc12_var                 0
mfcc13_mean                0
mfcc13_var                 0
mfcc14_mean                0
mfcc14_var                 0
mfcc15_mean                0
mfcc15_var                 0
mfcc16_mean                0
mfcc16_var                 0
mfcc17_mean                0
mfcc17_var                 0
mfcc18_mean                0
mfcc18_var                 0
mfcc19_mean                0
mfcc19_var                 0
mfcc20_mean                0
mfcc20_var                 0
dtype: int64
```

```
# Standardize variables
# MinMaxScaler for fixed range (0-1)
from sklearn.preprocessing import MinMaxScaler

minmax = MinMaxScaler()
minmax.fit(dfc)
standard_features = minmax.transform(dfc)

df_standard = pd.DataFrame(standard_features,columns=dfc.columns)
df_standard.head()
```

{"type":"dataframe","variable_name":"df_standard"}

```
# Perform descriptive statistics such as mean, median and mode
df_standard.describe()
```

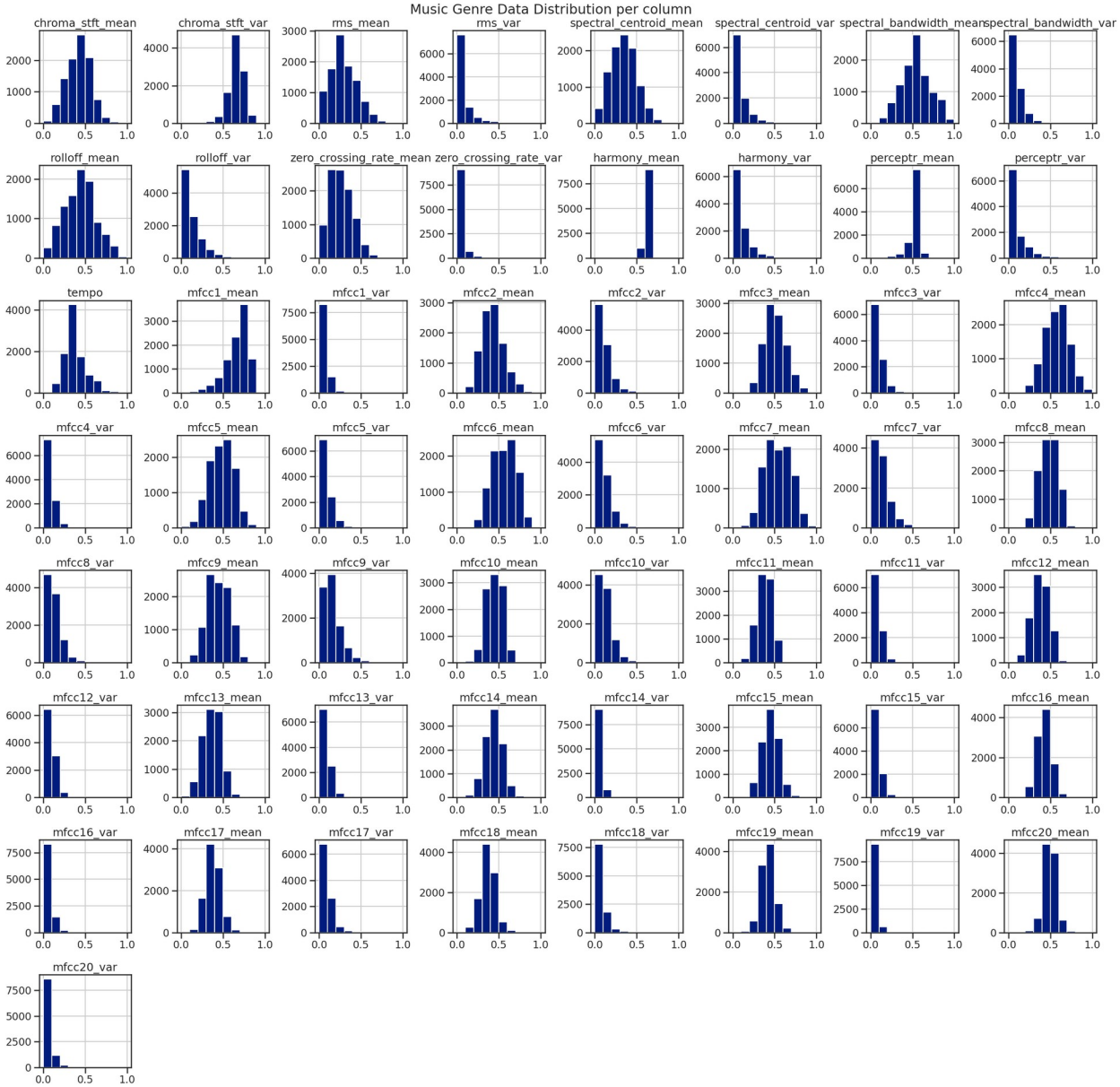
{"type":"dataframe"}

```
# Perform data visualization

# Distribution of each column
df_standard.hist(figsize=(25, 25), bins=10)

plt.suptitle("Music Genre Data Distribution per column")

plt.tight_layout()
plt.show()
```




```
# Build the model using K Nearest Neighbor. Use k value from 1 to 50.

# Evaluate the model using classification report, accuracy , confusion matrix and roc_auc_score

# Compare the model using their accuracy report.

# Create a graph to compare the accuracy based from eah k values.
```

Conclusion

In this activity, a K-Nearest Neighbors (KNN) classification model was successfully created and checked to predict binary class outcomes based on normalized tabular features. The KNN applies distance-based algorithms that are sensitive to feature magnitude. With this, standard

scaling was applied to equalize the variance across all independent variables. Evaluating the model made in the procedure returned a high classification accuracy of 91%. I was also able to apply the procedure in the supplementary activity, where I applied KNN in music genre classification.